

Jose Luis Serna.

Product Engineer · Solo builder

Construyo productos completos —de la base de datos a la última animación— con sensibilidad de diseño y criterio de sistema.



CREADOR DE

Nowdii · nowdii.com

La red social de los negocios locales

Zacualtipán, Hidalgo · México

CONTACTO

tamaa3295@gmail.com

nowdii.com

Abril 2026

01 · PERFIL

Qué hago — y por qué es raro encontrarlo

Diseño, programo y opero productos completos por mi cuenta. Construyo el frontend con la misma atención que el backend, tomo decisiones de producto sin esperar a que alguien me diga qué quiere, y documento lo que hago para que cualquiera pueda continuar sin mí. No soy "un desarrollador que también diseña" ni "un diseñador que programa" — soy el perfil intermedio que la mayoría de equipos pequeños necesita pero no siempre encuentra.

Mi especialidad es llevar productos de cero a producción sin intermediarios: esquema de base de datos, endpoints serverless, autenticación, PWA, diseño visual, animaciones, micro-copy, flujos de error, onboarding, panel de administración. Todo en el mismo repositorio, todo con criterio consistente, todo pensado para que el usuario sienta que está acompañado sin tener a nadie enfrente.

Trabajo mejor cuando hay un problema real sin resolver y recursos limitados. Ahí es donde el perfil full-stack solitario rinde: las decisiones se toman rápido porque no hay ping-pong entre roles, y la calidad no se diluye porque la misma persona que escribió la regla de seguridad en la base de datos es la que pintó el botón que la activa.

"Que el usuario sienta que está acompañado sin tener a nadie, solo por el simple hecho de que la interfaz está diseñada para no tener dudas."

— Principio de diseño que guía todo lo que construyo.

EN NÚMEROS

27 KLíneas de JS en
producción**8**

Endpoints serverless

14

Tablas en Supabase

192+

Versiones del SW

Todas son métricas del proyecto Nowdii, que construí y opero solo en paralelo al resto de mi vida profesional.

02 · STACK

Con qué construyo

El stack que uso es deliberadamente pequeño. Prefiero dominar pocas piezas a fondo que rotar librerías cada seis meses. La decisión de no usar frameworks de frontend en Nowdii no es dogma — es reconocer que en un proyecto que quiero operar solo durante años, cada dependencia nueva es un compromiso de mantenimiento futuro.

Frontend · cliente

HTML + CSS + JS (vanilla)	Sin framework. Componentes compartidos vía window.NowdiiX + tokens de diseño en CSS custom properties.
PWA	Service Worker con estrategia network-first + fallback offline + versionado manual para forzar actualización.
Leaflet	Mapa de descubrimiento con clustering y pines categorizados.
jsQR	Lectura de códigos QR en cámara del navegador (lazy-load desde CDN).
Librerías concretas cuando hacen falta	qrcode-generator, Resend, nada más.

Backend · plataforma

Supabase	Postgres + Auth (Google/Facebook/Email) + Storage + RPC + RLS. Migraciones SQL idempotentes versionadas en /docs/sql.
Vercel Serverless Functions	8 endpoints en Node: OG images, short links, canjes QR, validación de códigos digitales, email transaccional.
RLS con dos perfiles	Cientes usan la anon key con políticas restrictivas; endpoints críticos usan service_role con validación server-side explícita (ownership, anti-spoofing, geofencing).
Triggers + RPC en Postgres	Generación atómica de slugs únicos, códigos de 14 caracteres con alfabeto sin ambigüedad, contadores de canjes.

Herramientas de proceso

Git + GitHub	100+ commits en la última semana. Auto-deploy en push a main.
Documentación viva	Markdown en /docs: arquitectura, base de datos, visión, plan por etapas. Se actualiza en cada cierre de feature.
Demos HTML aislados	Cada rediseño grande se prototipa en un HTML auto-contenido con baseline + 3 opciones + comparativa antes de tocar producción.
Claude Code	Asistente de código para tareas repetitivas; las decisiones de producto las tomo yo.

03 · FORTALEZAS

Lo que me distingue

Estas cinco fortalezas no son autoreportadas — son observables en el repositorio de Nowdii. Cada una tiene evidencia concreta en código, documentación o decisiones de arquitectura.

01

Arquitectura pragmática

Sé qué NO usar. Elegí vanilla JS en lugar de React porque el proyecto está pensado para operarse solo durante años; cada dependencia nueva es deuda futura. El stack (Supabase + Vercel + PWA) está calibrado para que una sola persona pueda construir, desplegar y mantener sin fricción.

02

Ingeniería de sistema

Pienso en condiciones de carrera antes que en features. La Fase B6 (códigos digitales) usa reserva atómica vía PATCH condicional con rollback explícito si falla cualquier paso posterior. Anti-spoofing cruzando código con oferta. Fast-paths síncronos para el bug conocido de navigator.locks en Supabase JS v2.

03

UX con sensibilidad real

Tokens de diseño consistentes (colores, radios, tipografía). Tap feedback estilo app nativa. Modales con back-button y breadcrumb. Toasts en lugar de alert(). Estados vacíos con guía. Offline funcional. Cada microdetalle existe para reducir dudas.

04

Documentación como producto

NOWDII-ARCHITECTURE.md es un mapa de producción: qué tabla lee cada archivo, qué variables globales existen, en qué orden cargan los scripts, cuáles son las 8 conexiones críticas que no se pueden tocar sin entender. Cualquier dev puede continuar el proyecto sin mi intervención.

05

Proceso disciplinado

Antes de rediseñar la sección más densa del admin, construí un HTML auto-contenido con diagnóstico, baseline, tres opciones visuales y comparativa. Migro en pasos validados en producción ($A \rightarrow B \rightarrow C$ entre sesiones), nunca en un big-bang. Cada cierre incluye bump del Service Worker y entrada en el plan.

06

Visión de producto

Nowdii no es un directorio — es un sistema de confianza. La decisión de que seguir y guardar nunca gateen contenido es un principio estratégico, no una feature. Entiendo monetización (push orgánico vs pagado), growth (captación 1 a 1 antes que ads) y construyo en consecuencia.

04 · CASO DE ESTUDIO

Nowdii — La red social de los negocios locales

Nowdii es mi proyecto principal y la mejor evidencia de cómo trabajo. Llevo meses construyéndolo solo, en producción, con negocios reales, sin inversión externa y con la disciplina de un equipo de producto completo encarnado en una persona.

EL PROBLEMA

Para qué existe

En municipios medianos de México —como Zacualtipán, Hidalgo— los negocios locales viven fragmentados en internet. Facebook los mezcla entre memes y política. Google Maps tiene ubicaciones pero los perfiles son planos, sin vida, y posicionarse correctamente es tedioso. Instagram muestra fotos pero no genera confianza estructurada. TikTok hace viral el contenido pero no deja información útil. Los grupos de WhatsApp y Facebook son caóticos y las recomendaciones suelen venir del propio dueño o sus conocidos.

El cliente termina preguntando "¿dónde llevo mi carro al mecánico?" en un grupo de 15.000 personas y recibe 15 respuestas de conocidos del mecánico. No confía en esa información y lo sabe.

LA VISIÓN

Qué estoy construyendo

Nowdii es el espacio digital donde los negocios muestran su identidad completa —quiénes son, qué ofrecen, cómo se ven, dónde están, qué tan confiables son— de forma ordenada, verificada y permanente. Es a los negocios locales lo que LinkedIn es a los profesionales: el lugar donde se construye credibilidad real, no solo presencia.

"Manana alguien dirá: oye Nowdii, quiero ir a Zacualtipán, ¿dónde puedo comer, cuáles son los mejor calificados, dónde atienden más rápido? Y Nowdii lo resuelve en segundos porque tiene los datos limpios y verificados que otros directorios no tienen."

La apuesta estructural es que el activo no es el código —ese lo puede copiar cualquiera— sino los datos verificados que la comunidad aporta. Cuando la IA alcance madurez para responder búsquedas conversacionales, Nowdii ya tendrá la base de datos más confiable de la región porque otros directorios aceptaron todo sin criterio.

04 · CASO DE ESTUDIO

Arquitectura — cómo está construido

Nowdii corre sobre tres capas: un frontend vanilla servido como PWA, una plataforma Supabase para datos y auth, y endpoints serverless en Vercel para operaciones críticas que no pueden ejecutarse con anon key. Todo el código vive en un solo repositorio privado y se despliega automáticamente en cada push a main.

Capa de datos

Postgres en Supabase con 14 tablas principales: negocios, servicios, usuarios, categorías, intenciones de búsqueda, ofertas, anuncios, aportaciones, fotos, historias, favoritos, seguidores, ofertas guardadas, canjes, códigos digitales. RLS habilitado en todas las tablas sensibles, con políticas específicas por rol. Funciones RPC escritas en plpgsql para operaciones atómicas (generación de slugs únicos, comprobantes de canje, códigos NOW-XXXX-YYYYYY). Migraciones SQL idempotentes en /docs/sql para que se puedan re-ejecutar sin miedo.

Capa de búsqueda

Sistema de dos niveles: la capa server-side usa un RPC que consulta primero una tabla de intenciones de búsqueda curada (ej. "comida rápida" → categoría alimentos + subcategoría rápida) y cae en Full-Text Search si no hay match. La capa client-side es un fallback con scoring local que usa un diccionario de sinónimos, stopwords e intenciones hardcodeadas. Ambas capas se pueden mantener sincronizadas desde el panel admin.

Capa de aplicación

Módulos globales compartidos (window.NowdiiFlyer, window.NowdiiScanner, window.NowdiiCodigoModal, window.NowdiiShare, window.nToast, window.nConfirm) que encapsulan flujos completos y se pueden invocar desde cualquier página. Cada HTML tiene su CSS/JS dedicado con el mismo nombre, más un set de archivos compartidos claramente marcados. El orden de carga está documentado por página.

Capa serverless

Ocho endpoints en Vercel. Los simples (og.js, c.js, notificar-admin.js) no manejan auth. Los críticos (canje.js, codigo-validar.js, codigo-generar.js, canje-confirmar.js) validan Bearer JWT, cruzan ownership contra la tabla usuarios, y usan el service_role para operaciones que saltan RLS con validación explícita en código.

Capa de confianza

Badges de verificación en 3 niveles (Azul automático, Dorado con 50+ reseñas positivas, Plateado con 200+). Sistema de reputación por atributos en beta: en lugar de estrellas promedio, cada negocio se pinta con el atributo que más destaca (Rápido / Sabor / Precio justo / Atención / Limpio) según las aportaciones reales de usuarios verificados.

04 · CASO DE ESTUDIO

Fase B — sistema de canjes en dos canales

Para ilustrar el nivel de detalle con el que construyo, lo que hice el fin de semana del 19 de abril es un buen ejemplo. En dos días cerré una fase completa que incluye dos flujos paralelos de canje de ofertas, dos tablas nuevas, cuatro endpoints y los componentes UI necesarios para que el usuario no dude qué hacer.

EL RETO

Un mismo concepto, dos realidades

Un canje es el momento en que un cliente reclama una oferta al negocio. Pero no todos los negocios son iguales: un restaurante tiene un local físico donde el cliente puede escanear un QR al llegar; un fotógrafo, un músico o un servicio a domicilio no. Si obligo a todos a usar QR físico dejo fuera a la mitad de los giros. Si solo uso códigos digitales elimino la validación geográfica que protege al negocio con local contra fraude remoto.

LA SOLUCIÓN

Dos canales, una misma tabla

Diseñé dos flujos independientes que escriben en la misma tabla **canjes** con un campo **canal** que diferencia el origen. El negocio ve todos sus canjes unificados en el panel y confirma/rechaza con el mismo botón.

Canal qr_fisico — para negocios con local

El cliente toca "Canjear ahora" en el flyer de la oferta. Se abre un modal fullscreen con la cámara. jsQR detecta el QR del mostrador (una URL `/c/<slug>`). Se pide geolocalización y se hace POST al endpoint. El servidor valida Bearer JWT, busca el negocio por el slug del QR, calcula Haversine contra las coordenadas del usuario (radio configurable por variable de entorno), verifica UNIQUE por user+oferta, genera un comprobante de 5 caracteres vía RPC e inserta el canje en estado "presunto" con `service_role`.

Canal codigo_digital — para negocios sin local

El operador entra a su panel, selecciona una oferta activa y pulsa "Generar código". El endpoint crea un código NOW-XXXX-YYYYY con 30 minutos de vigencia (alfabeto sin ambigüedad: sin I, L, O, 0, 1). El operador copia el código y lo manda por WhatsApp al cliente. El cliente toca "Tengo un código" en el flyer, lo teclea (con autoformat) y se valida.

La parte difícil — reserva atómica y rollback

El endpoint de validación es lo que mejor muestra cómo pienso. Dos clientes podrían intentar usar el mismo código al mismo tiempo. Un SELECT + INSERT secuencial abriría una ventana de carrera. En lugar de eso, el flujo cierra en cinco pasos controlados:

- 1. Reserva atómica.** PATCH condicional: actualiza *usado_at=ahora* solo si *usado_at IS NULL* y *expira_at > NOW()*. Si el UPDATE no afecta ninguna fila, otro cliente se adelantó — se devuelve el error correcto (CODIGO_USADO / CODIGO_EXPIRADO).
- 2. Anti-spoofing.** Se verifica que el código corresponde exactamente a la oferta que el cliente dice estar canjeando. Sin este paso, alguien podría teclear un código generado para otra oferta del mismo negocio.
- 3. Generación de comprobante.** RPC que devuelve 5 caracteres únicos. Si falla se libera el código (PATCH de rollback).
- 4. INSERT del canje** con *service_role* y *Prefer: return=representation*. Si el insert falla por UNIQUE (el user ya canjeó esta oferta) se libera el código y se devuelve YA_CANJEADO.
- 5. Vinculación código ↔ canje** en best-effort. Si falla no bloquea la respuesta porque el canje ya quedó registrado.

El resultado: el endpoint nunca deja un código reservado sin canje asociado, nunca inserta un canje sin liberar el código si falla el resto, y nunca permite doble uso.

04 · CASO DE ESTUDIO

UX — diseñar para que nadie tenga dudas

Los sistemas arquitectónicos del caso anterior no valen nada si el usuario no entiende qué hacer. En Nowdii el diseño y la ingeniería son la misma disciplina. Cada detalle visible está pensado para reducir la fricción cognitiva.

Lenguaje de diseño consistente

Tokens CSS en *:root* (colores, radios, tipografía) compartidos por todas las páginas. Un mismo chip se pinta igual en la card, en el flyer, en el admin y en el editor. Un mismo badge de verificación se renderiza con el mismo SVG en 5 páginas distintas vía *verificado-icon.js*. Los toasts amarillos reemplazan los *alert()* nativos en todos los flujos.

Feel de app nativa

Tap feedback estilo iOS/Android con *scale(0.96)* en botones, tipografía con pesos 800 en títulos y 700 en nombres de card (inspirado en IG/TikTok), scroll suave global, *will-change* en heros con parallax, PWA instalable con manifest y pantalla offline custom.

Modales con jerarquía clara

Cada modal tiene un header con tres piezas: botón atrás con flecha, breadcrumb en caps (para que el usuario sepa dónde está dentro del flujo) y título de la acción. El botón cerrar vive siempre en el mismo lugar. Esto se aplica igual en el panel admin (que solo veo yo) que en el flyer público — porque la consistencia paga ella sola.

Estados vacíos con guía

Cuando un negocio abre su panel y no tiene ofertas aún, no ve una pantalla en blanco sino una guía con los 4 tipos de oferta disponibles (Estándar, Cupón, Combo, Relámpago) y cómo elegir entre ellos. Cada tipo tiene color y comportamiento propio.

Manejo de errores con mensajes humanos

Los endpoints devuelven códigos semánticos (CODIGO_USADO, CODIGO_EXPIRADO, YA_CANJEADO, OFERTA_INACTIVA) y mensajes redactados para ser leídos por personas: "Este código ya fue usado" / "Pídele al negocio uno nuevo" — en lugar de "error 400".

Workflow de rediseño con opciones comparadas

Antes de rediseñar una pantalla densa (como el panel del operador con 14 tabs y 6+ modales), construyo un HTML auto-contenido que incluye diagnóstico de los puntos de dolor, baseline actual, tres variantes A/B/C con diferentes niveles de reescritura, y comparativa tabular. Después decido con el usuario (en este caso yo mismo) cuál aplicar y migramos en pasos entre sesiones, validando en producción antes de seguir. Este proceso evita re-trabajo y deja evidencia de las decisiones tomadas.

05 · PROCESO

Cómo trabajo

Mi proceso está calibrado para que el producto avance rápido sin acumular deuda. Cada feature sigue un ciclo claro que documenté con el tiempo y sigo aplicando.

Diagnóstico

Antes de escribir código, identifico los puntos de dolor reales. Si es un rediseño, listo los problemas concretos observables; si es una feature nueva, empiezo por "qué cambia en la experiencia del usuario" y después por "qué tabla toca".

Baseline + opciones

Para cambios grandes, construyo un HTML aislado con la versión actual, 2-3 variantes y una comparativa de trade-offs. Esto evita empezar a programar la opción equivocada.

Implementación en pasos

Divido en pasos chicos que se pueden validar en producción. Nunca un big-bang. Cada paso bumpea el Service Worker y se documenta.

Validación en campo

Uso el producto yo mismo en el teléfono, con el navegador abierto al mismo tiempo para inspeccionar. Pruebo en datos móviles para confirmar el perfil de carga.

Cierre + documentación

Cada feature cerrada entra en NOWDII-PLAN.md bajo "Ya en producción" con la fecha, una nota del impacto y los archivos tocados. Si abre pendientes, se documentan.

Criterio sobre cantidad

No todas las ideas que se me ocurren se implementan. Aplico la regla "¿esto resuelve un problema real observable?" antes de escribir nada. En Nowdii tengo una lista de pendientes reales y una lista separada de mejoras incrementales para cuando hay espacio.

06 · PARA TU EMPRESA

Qué puedo aportar

Si tu empresa quiere desarrollar un sistema, la pregunta no es si el perfil técnico alcanza — es si alguien puede cubrir varias funciones sin que se diluya la calidad de ninguna. Aquí es donde rindo.

Lo que puedo hacer desde el día uno

- **Diseñar el esquema de base de datos.** Relaciones, índices, políticas RLS, triggers y RPCs en Postgres. Migraciones idempotentes versionadas.
- **Construir el backend.** Endpoints serverless en Node con validación de auth, ownership cruzado y manejo explícito de concurrencia. Integraciones (Stripe, Resend, SendGrid, WhatsApp Cloud API) donde apliquen.
- **Desarrollar el frontend.** Desde sistemas de diseño propios hasta animaciones custom. Vanilla JS cuando el proyecto lo pide; React/Next si el equipo ya lo usa.
- **Diseñar la experiencia completa.** Flujos de onboarding, paneles de administración, estados vacíos, mensajes de error humanos, micro-copy, tap feedback.
- **Documentar todo.** Arquitectura como mapa, no como README decorativo. Cualquier persona que entre después puede continuar el trabajo.
- **Operar producción.** Deployments, monitoreo, versionado de caches, debugging remoto, manejo de incidentes. He operado Nowdii solo durante meses.

Lo que me motiva trabajar

- Problemas reales sin resolver, no proyectos internos para llenar tiempo.
- Equipos pequeños donde las decisiones se toman rápido y la persona que programa puede discutir la estrategia.
- Sistemas que van a estar en manos de usuarios no-técnicos y necesitan no tener dudas.
- Contextos donde el trade-off entre velocidad y calidad se resuelve con criterio, no con procesos de una empresa grande aplicados a un equipo de tres.
- Stacks dominables. Prefiero 5 herramientas bien entendidas a 20 montadas sin razón.

Lo que no pretendo ser

- No soy especialista profundo en ML/IA. Uso modelos como herramientas; no entreno.
- No soy DevOps ni SRE dedicado. Sé operar producción, pero no construyo infraestructura a escala Netflix.
- No soy gerente de producto corporativo. Pienso como producto pero ejecuto; no coordino equipos de 30.
- Soy honesto sobre estas líneas. Lo que sí hago, lo hago bien.

07 · CONTACTO

Hablemos

Si lo que leíste tiene sentido para lo que tu empresa está construyendo, mándame un mensaje. Me gusta escuchar antes de proponer: cuéntame qué están intentando resolver, qué intentos previos hubo y dónde se atorán hoy. Con eso puedo responder si encajo y, si encajo, cómo empezaríamos.

Email	tamaa3295@gmail.com
Proyecto vivo	nowdii.com
Base	Zacualtipán, Hidalgo · México
Disponibilidad	Full-time remoto, híbrido regional o proyectos por hito
Código fuente	Disponible para revisión bajo acuerdo de confidencialidad

LO QUE SIGUE

Una conversación breve

Con una llamada de 30 minutos tengo suficiente para decidir si el proyecto me entusiasma y si creo que puedo aportar valor real. No me interesa convencer a nadie de contratarme — prefiero que ambas partes lleguen a la conclusión correcta sin filtros.

Jose Luis Serna · Product Engineer

Documento generado el 20 de abril de 2026 · Nowdii v193